Arquitectura de LA TROUPE — v3.3

Marketplace de formación en danza: cursos propios, profesores invitados y contenido para academias

Documento de cambios y decisiones sobre la v3.2. Fecha: octubre de 2026.

> **Cómo leer este documento.** La v3.3 no repite las tablas que no cambian: todo lo que no se menciona aquí sigue como en la v3.2 (`Arquitectura_LA_TROUPE_v3.2.md`). Esta versión **reemplaza** lo indicado en la sección 1.3 y **agrega** los dominios nuevos. Cuando haya una contradicción, manda la v3.3.

---

1. Qué cambió y por qué

1.1 El producto es más grande que lo que describía la v3.2

La v3.2 modelaba una academia que vende cursos propios. Lo que definimos después es un **marketplace con tres públicos y tres tipos de autor**:

Dimensión	Valores
Quién crea el contenido	La academia (LA TROUPE); profesores propios o invitados que venden bajo la marca de la academia; (a futuro) otras academias en su propio espacio
Quién consume	Alumnos; profesores que se especializan; academias que usan el contenido para dar sus clases presenciales
Cómo se cobra	Pago único; cuotas con tarjeta; suscripción recurrente; plan de pagos con acceso condicionado a estar al día
Cómo se avanza	Libre (todo desbloqueado) o curricular (se libera por tiempo, por examen aprobado, o por ambos)
Cómo se paga al autor	Reparto automático por porcentaje; o la academia cobra todo y paga un monto acordado

1.2 Principio rector de la v3.3

> **Decidir el modelo hoy, construir las pantallas cuando toque.**

> Lo que es caro de cambiar después (columnas en tablas existentes, relaciones, quién es "titular" de un acceso) se reserva desde el día uno. Lo que es independiente (una tabla nueva que no toca a las demás) se crea cuando empieza su fase.

Este principio completa al 2.10 de la v3.2 (captura de datos desde el día uno) y reemplaza el uso de YAGNI como argumento para dejar fuera cambios estructurales.

1.3 Qué reemplaza esta versión

Elemento de la v3.2	Qué pasa en la v3.3
`memberships.system_role / domain_role / staff_role` (tres enums)	Se reemplazan por roles y permisos editables (sección 3)
Jerarquía solo curso → lección	Pasa a carrera → curso → lección → bloques (sección 4)
`course_lessons.is_locked_until` (días fijos)	Se reemplaza por reglas de liberación (sección 5)
`access` solo para perfiles de alumno	`access` admite titular alumno **u organización** (sección 8)
Compras vinculadas directamente a cursos	Se vende un **producto**, que puede contener cursos o carreras (sección 8)

Elemento de la v3.2	Qué pasa en la v3.3
Reparto: solo `teacher_payouts` (sueldo por clase)	Se agregan acuerdos de reparto y división de pagos (sección 9). `teacher_payouts` se mantiene para el pago manual
Fase 1 incluía "access sin entitlements" y Stripe en Fase 2	Roadmap rehecho (sección 11)
Cuenta de 24 tablas en la portada / 27 en la sección 3 / `staff_profile` descartada en 4.5 y creada en 4.10	Se unifica: `staff_profile` **existe** (4.10 manda); el conteo se actualiza en la sección 12

---

2. Principios (v3.2 más los nuevos)

Siguen vigentes los principios 2.1 a 2.10 de la v3.2. Se agregan:

2.11 Reglas como datos, también para el avance, la evaluación y el egreso

Cómo se libera el contenido, cómo se aprueba un examen y cuándo alguien egresa **no se programan por curso**. Se describen como reglas guardadas en tablas que cada curso o carrera configura. Así, un curso nuevo con una modalidad que hoy no imaginamos se crea desde el panel, sin cambiar código.

2.12 El contenido se vende a través de productos

Un curso es contenido. Un **producto** es lo que se vende (con su precio, su modalidad de cobro y su política de acceso). Un producto puede contener un curso, una carrera o un paquete. Esto permite cambiar precios, armar paquetes y vender en varias monedas sin tocar el contenido.

2.13 El titular de un acceso puede ser una persona o una organización

Esto habilita vender contenido a academias (licencias) sin crear un segundo sistema.

2.14 Dinero: cada cobro es inmutable y cada reparto queda registrado

Un pago nunca se edita. Si hay reembolso o ajuste, se agrega un registro nuevo. El reparto entre academia, profesor y comisiones se registra por cada pago, de modo que cualquier reporte se pueda reconstruir.

2.15 Aprobado significa congelado

El contenido que un profesor sube pasa por revisión. Una vez aprobado queda fijo. Cualquier cambio posterior se pide como solicitud y alguien con permiso la aprueba.

---

3. Dominio: Identidad, roles y permisos (cambia)

3.1 Por qué se reemplazan los tres enums

Con `system_role`, `domain_role` y `staff_role` fijos, cada tipo de empleado nuevo requería cambiar el código, y una persona que fuera a la vez profesor y alumno quedaba mal representada. Además, en el modelo SaaS cada academia definirá sus propios niveles de permiso.

3.2 Tablas nuevas

`permissions` (catálogo global, mantenido por el equipo de desarrollo). Cada permiso es una acción sobre un recurso, con un alcance.

Campo	Tipo	Notas
id	UUID	Primary key
key	string	Ej: `course.edit`, `course.publish`, `student.view`, `payment.view`, `grade.assign`
scope	enum	`own` (solo lo propio), `org` (toda la organización)
description	text	Texto para mostrar en el panel de roles

`roles` (por organización, con plantillas de sistema).

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	Null para las plantillas de sistema
name	string	Ej: 'Profesor invitado', 'Cobrador', 'Coordinadora académica'
is_system_template	boolean	True para los roles que vienen de fábrica
is_editable	boolean	Los roles de sistema críticos (owner) no se pueden modificar

`role_permissions`: tabla pivote `role_id` + `permission_id`.

`membership_roles`: tabla pivote `membership_id` + `role_id`. Una persona puede tener **varios roles** en una misma organización (profesor y alumno, por ejemplo).

3.3 Cambios en `memberships`

Se eliminan `system_role`, `domain_role` y `staff_role`. Quedan: `user_id`, `organization_id`, `location_id` (opcional), `status`. Los roles viven en `membership_roles`.

3.4 Roles de fábrica (plantillas)

Rol	Puede
Owner	Todo en su organización, incluida la facturación
Administrador	Todo menos borrar la organización y cambiar al owner
Profesor	Crear y editar **sus** cursos en borrador; ver los alumnos de **sus** cursos; calificar
Profesor invitado	Igual que Profesor, pero sus cursos requieren aprobación para publicarse y sus cambios requieren solicitud
Revisor de contenido	Aprobar o rechazar cursos y solicitudes de cambio
Finanzas	Ver pagos, repartos y reportes de ingresos
Atención al alumno	Ver alumnos, accesos y pagos; no editar contenido
Alumno	Ver los cursos a los que tiene acceso

La academia puede clonar y modificar estos roles. El panel de la plataforma (Fase 6) podrá agregar plantillas nuevas.

3.5 Cómo se chequea un permiso

Se resuelve en una sola función del backend (y, donde corresponda, en una política de Row-Level Security): *¿esta membresía tiene un rol con el permiso `X` y, si el alcance es `own`, el recurso pertenece a este usuario?* No se dispersan chequeos de rol por la aplicación.

---

4. Dominio: Catálogo y contenido (cambia)

4.1 Jerarquía

```
Carrera (program)
  └ Curso / módulo (course)
       └ Lección (lesson)
            └ Bloques (lesson_blocks): video, texto, diapositivas, descarga
```

Un curso puede existir **solo** (sin carrera) o dentro de una o varias carreras. Una lección sigue siendo atómica y global, como dice el principio 2.3.

4.2 Tabla nueva `programs` (carrera)

Campo	Tipo	Notas
id	UUID	Primary key
organization_id	UUID	
title	string	Ej: 'Formación en Urbano — Año 1'
description	text	
pacing_mode	enum	`self_paced` (libro abierto) o `curricular` (se libera según reglas)
duration_months	integer	Informativo, para mostrar al alumno
cover_asset_id	UUID	Imagen de marketing
trailer_asset_id	UUID	Tráiler corto
status	enum	draft, published, archived

`program_courses` (pivote): `program_id`, `course_id`, `position`. Las reglas de cuándo se abre cada curso viven en `release_rules` (sección 5).

4.3 Cambios en `courses`

Se agregan:

Campo	Tipo	Notas
author_user_id	UUID	Quién lo creó o es responsable (profesor)
owner_type	enum	`academy_owned`, `teacher_owned`, `licensed`
audience	string[]	`student`, `teacher_training`, `academy_b2b`. Un curso puede apuntar a más de un público
review_status	enum	`draft`, `in_review`, `changes_requested`, `approved`, `published`, `unpublished`
approved_by	UUID	Quién lo aprobó
approved_at	timestamp	Desde esa fecha el contenido está congelado
pacing_mode	enum	Igual que en `programs`, para cursos que se venden solos
trailer_asset_id	UUID	Tráiler del curso
hover_preview_asset_id	UUID	Clip de 15 a 60 s que se reproduce al pasar el mouse por la tarjeta

Clip al pasar el mouse. Es público: lo ven los visitantes y los que no compraron. Bunny.net Stream genera vistas previas animadas de cada video; hay que **verificar en la fase técnica** si alcanzan para el efecto buscado o si conviene subir un clip editado aparte. El modelo soporta las dos opciones, porque el clip es un asset más.

4.4 Cambios en `lessons` y bloques de contenido

Una lección ya no es "un video". Es un contenedor ordenado de **bloques**.

`lesson_blocks` (nueva)

Campo	Tipo	Notas
id	UUID	
lesson_id	UUID	
position	integer	Orden dentro de la lección
block_type	enum	`video`, `rich_text`, `slides`, `download`, `quiz_link`
body	text	Solo para `rich_text`
asset_id	UUID	FK a `lesson_assets` (video, PDF, diapositivas)
is_free	boolean	Visible sin comprar

`lesson_assets.asset_type` agrega: `document`, `slides`, `trailer`, `hover_preview`, `attachment`. Todo lo demás de `lesson_assets`, `lesson_chapters`, `lesson_progress` y `lesson_video_views` sigue igual.

`lessons` agrega `author_user_id` y hereda el estado de revisión del curso al que pertenece.

4.5 Revisión y solicitudes de cambio

`course_change_requests` (nueva)

Campo	Tipo	Notas
id	UUID	
course_id	UUID	
requested_by	UUID	Profesor que pide el cambio
change_type	enum	`edit`, `remove`, `add`, `unpublish`
target_type / target_id	string / UUID	Qué se quiere tocar (lección, bloque, etc.)
proposed_payload	JSON	El cambio propuesto
reason	text	
status	enum	`pending`, `approved`, `rejected`, `applied`
reviewed_by / reviewed_at	UUID / timestamp	

Reglas:

- Mientras el curso está en `draft` o `changes_requested`, el profesor edita libremente.
- Desde `approved`, la edición directa queda bloqueada para quien tenga el rol *Profesor invitado*. Los roles con permiso `course.edit` en alcance `org` (administradores) sí pueden editar.
- Cada solicitud aprobada deja registro en `event_log`.

---

5. Dominio nuevo: Progresión y liberación de contenido

Este dominio es una de las piezas diferenciales del producto. Soporta, con las mismas tablas, estas modalidades:

Modalidad	Cómo se expresa
Libro abierto	`pacing_mode = self_paced`: todo está desbloqueado desde la compra
Curricular por tiempo	Cada curso o lección se abre N días (o N meses) después de la inscripción **de ese alumno**
Curricular por examen	Se abre cuando el alumno aprueba una evaluación determinada

Modalidad	Cómo se expresa
Curricular combinado (todas)	Se abre solo si pasó el tiempo **y** aprobó el examen
Curricular combinado (alguna)	Se abre si pasó el tiempo **o** aprobó el examen

Cada alumno corre con su propio reloj: quien se inscribe en enero y quien se inscribe en junio ven el mismo contenido en el mismo orden, pero a su ritmo y a partir de su fecha de inicio.

5.1 `learning_enrollments` (nueva)

La inscripción de un alumno a un producto de aprendizaje. **Es la fecha de referencia de todos los cálculos de tiempo.** (No confundir con `class_enrollments`, que son las inscripciones a clases en vivo de la Fase 3.)

Campo	Tipo	Notas
id	UUID	
organization_id	UUID	
student_profile_id	UUID	
program_id / course_id	UUID	Uno de los dos
source_access_id	UUID	Qué acceso la originó
started_at	timestamp	Cuándo arrancó el reloj. Normalmente la fecha de compra; el admin puede moverla
status	enum	`active`, `paused`, `completed`, `dropped`
paused_at	timestamp	Si se pausa, el reloj se detiene

5.2 `release_rules` (nueva)

Una regla por cada elemento que se desbloquea (curso dentro de una carrera, o lección).

Campo	Tipo	Notas
id	UUID	
organization_id	UUID	
target_type	enum	`program_course`, `lesson`
target_id	UUID	
combine	enum	`all` (se cumplen todas las condiciones) o `any` (alcanza una)
conditions	JSON	Lista de condiciones. Ver ejemplos abajo
created_at	timestamp	

Tipos de condición:

Tipo	Parámetros	Significa
`time_since_enrollment`	`months` o `days`	Pasó este tiempo desde `learning_enrollments.started_at`
`assessment_passed`	`assessment_id`, `min_score`	Aprobó esta evaluación
`previous_completed`	`target_id`	Completó el contenido anterior
`fixed_date`	`date`	Fecha de calendario (útil para cohortes con fecha común)

Ejemplos:

- *Módulo 2 se abre al mes y exige aprobar el examen del módulo 1*: `combine = all`, condiciones `[time_since_enrollment: 1 mes, assessment_passed: examen_modulo_1]`.
- *Módulo 3 se abre al mes o al aprobar el examen, lo que ocurra primero*: `combine = any`.

- *Todo abierto*: sin reglas, o `pacing_mode = self_paced`.

Por qué JSON y no columnas: la lista de tipos de condición va a crecer. El JSON se valida contra un esquema (Zod en el backend, `CHECK` en la base) y se puede consultar. Si una condición llega a usarse en reportes masivos, se promueve a tabla propia sin afectar al resto.

5.3 `content_unlocks` (nueva)

Registro **persistente** de qué desbloqueó cada alumno y por qué.

Campo	Tipo	Notas
id	UUID	
learning_enrollment_id	UUID	
target_type / target_id	enum / UUID	
unlocked_at	timestamp	
reason	enum	`time`, `assessment`, `manual_override`, `always_open`
unlocked_by	UUID	Solo si fue un desbloqueo manual

Qué resuelve:

- Si la academia cambia una regla más adelante, **lo ya desbloqueado no se vuelve a bloquear**.
- Un admin o profesor puede desbloquear manualmente a un alumno en particular (por ejemplo, tras un reclamo) y queda registrado.
- Un trabajo programado diario (`pg_cron`) evalúa las reglas de tiempo y escribe los desbloqueos nuevos. Las reglas de examen se evalúan al calificar.

5.4 Función central de acceso

`canUserAccessLesson` (v3.2, sección 5.4) pasa a ser:

1. Si la lección es gratuita (`is_free`) → SÍ.
2. Si el alumno no tiene un `access` activo que cubra ese curso → NO (mostrar compra).
3. Si el `access` está pausado por falta de pago (sección 8.4) → NO (mostrar regularizar pago).
4. Si el curso es `self_paced` → SÍ.
5. Si existe un registro en `content_unlocks` → SÍ.
6. En cualquier otro caso → NO (mostrar cuándo y cómo se desbloquea).

Esta función es **la única** que decide, y se usa tanto desde la interfaz como desde la protección del video.

---

6. Dominio nuevo: Evaluación, calificación y egreso

6.1 Evaluaciones

`assessments`

Campo	Tipo	Notas
id	UUID	
organization_id	UUID	
course_id / lesson_id	UUID	A qué contenido pertenece
title	string	

Campo	Tipo	Notas
grading_type	enum	`auto` (corrección automática), `manual` (la corrige un docente), `mixed`
passing_score	decimal	Nota mínima para aprobar
max_score	decimal	
max_attempts	integer	Null = ilimitados
availability	JSON	Cuándo se puede rendir. Ej: `{ "type": "monthly_window", "day_from": 1, "day_to": 7 }` para "una vez al mes por módulo"
time_limit_minutes	integer	Opcional
created_by	UUID	

`assessment_questions`: `assessment_id`, `position`, `question_type` (`multiple_choice`, `true_false`, `short_text`, `file_upload`, `video_upload`), `prompt`, `options` (JSON), `correct_answer` (JSON, null si la corrige un docente), `points`.

`assessment_attempts`

Campo	Tipo	Notas
id	UUID	
assessment_id	UUID	
student_profile_id	UUID	
started_at / submitted_at	timestamp	
auto_score	decimal	Parte corregida por el sistema
manual_score	decimal	Parte puesta por el docente
final_score	decimal	Suma, calculada al terminar la corrección
status	enum	`in_progress`, `submitted`, `pending_review`, `graded`
passed	boolean	Resultado contra `passing_score`
graded_by	UUID	Docente que corrigió (si aplica)
feedback	text	Devolución general

`attempt_answers`: `attempt_id`, `question_id`, `answer` (JSON), `asset_id` (si subió archivo o video), `auto_points`, `manual_points`, `feedback`.

Flujo `mixed`: el sistema corrige lo automático al enviar; el intento queda en `pending_review`; el docente completa la parte manual; recién ahí `final_score` y `passed` se fijan y se disparan los desbloqueos que dependan de ese examen.

6.2 Reglas de egreso

`completion_rules` (una por curso o carrera)

Campo	Tipo	Notas
id	UUID	
target_type / target_id	enum / UUID	`course` o `program`
criteria	JSON	Combinación de criterios. Ver abajo
requires_manual_approval	boolean	Si un docente o admin debe aprobar el egreso
issues_certificate	boolean	
certificate_template_id	UUID	

Criterios disponibles:

Criterio	Parámetros
`all_lessons_completed`	`min_percentage` (por defecto 85, como en `lesson_progress`)
`min_average_grade`	`value`
`assessments_passed`	`which`: `all` o lista de ids
`all_courses_completed`	Solo para carreras

Combinando criterios y `requires_manual_approval` se cubren los tres casos pedidos: egreso automático al completar y aprobar; egreso mixto con visto bueno del docente; egreso decidido por el docente (sin criterios automáticos, solo aprobación manual).

6.3 Egresos y certificados

`completions`

Campo	Tipo	Notas
id	UUID	
student_profile_id	UUID	
target_type / target_id	enum / UUID	
status	enum	`in_progress`, `pending_approval`, `completed`, `rejected`
final_grade	decimal	
approved_by	UUID	Si hubo aprobación manual
completed_at	timestamp	

`certificate_templates` (por organización): nombre, diseño (asset), campos variables (nombre, curso, fecha, nota, firmante), firmante.

`certificates`

Campo	Tipo	Notas
id	UUID	
completion_id	UUID	
certificate_number	string	Correlativo legible
verification_code	string	Código único para validar autenticidad
issued_at	timestamp	
pdf_asset_id	UUID	El PDF generado queda guardado, no se regenera
revoked_at / revoked_reason	timestamp / text	Un certificado puede revocarse sin borrarse

Se publica una **página pública de verificación** (`/verificar/<código>`) que confirma nombre, curso y fecha sin exponer otros datos del alumno.

---

7. Calificaciones: visibilidad

- El alumno ve sus notas y devoluciones.
- El profesor ve las de **sus** cursos (permiso `grade.assign`, alcance `own`).
- Los administradores ven todas (alcance `org`).
- El menor de edad: las notas se muestran a quien administra el perfil (`account_user`).

---

8. Dominio: Catálogo comercial y accesos (cambia)

8.1 Productos

`products`

Campo	Tipo	Notas
id	UUID	
organization_id	UUID	
name	string	Lo que ve el cliente
product_type	enum	`course`, `program`, `bundle`, `license`
audience	string[]	Para filtrar el catálogo
status	enum	draft, active, archived

`product_items`: `product_id`, `item_type` (`course`, `program`), `item_id`. Un producto puede contener varios.

8.2 Precios y modalidades de cobro

`prices` (un producto puede tener varios precios: por moneda, por país, por modalidad)

Campo	Tipo	Notas
id	UUID	
product_id	UUID	
currency	string	ISO 4217: UYU, USD, EUR, ARS…
amount	decimal	
tax_inclusive	boolean	Reserva para IVA/impuestos
billing_model	enum	Ver tabla abajo
interval	enum	monthly, quarterly, yearly (solo suscripción/plan)
installments_count	integer	Cantidad de cobros del plan, o de cuotas con tarjeta
access_policy	enum	`lifetime`, `while_paid`, `fixed_term`
access_term_days	integer	Solo si `fixed_term`
allowed_countries	string[]	Null = todos
gateway_preference	enum	Opcional: forzar un gateway

Las cuatro modalidades pedidas se expresan así:

Modalidad	`billing_model`	`access_policy`	Quién la ejecuta
Pago único	`one_time`	`lifetime`	Gateway
Cuotas con tarjeta	`one_time` + `installments_count` en el checkout	`lifetime`	**El gateway** (MercadoPago/Stripe): la academia cobra el total y el banco financia
Suscripción recurrente	`subscription`	`while_paid`	Gateway (cobro recurrente)
Plan de pagos propio (N cobros mensuales)	`payment_plan`	`while_paid` o `lifetime` al terminar	**Nuestro sistema**: genera N facturas con vencimiento; cobra cada una

Una **suscripción** es un plan sin fin. Un **plan de pagos** es la misma estructura con `cycles_total` fijo. Por eso no hay tablas distintas: `subscriptions` (v3.2) agrega `cycles_total` (nullable) y `cycles_paid`.

8.3 Cambios en `access`

Campo	Cambio
student_profile_id	**Pasa a nullable**
grantee_type / grantee_id	**Nuevos.** `student_profile` u `organization`. Permite licencias a academias
product_id	**Nuevo.** Qué producto originó el acceso
status	**Nuevo.** `active`, `paused_unpaid`, `expired`, `revoked`
requires_payment_current	**Nuevo.** Boolean, copiado desde `prices.access_policy = while_paid`
valid_until	Se mantiene (`expires_at`)

8.4 Acceso condicionado al pago

Cuando el acceso depende de estar al día:

1. Un cobro falla → la factura queda vencida.
2. Pasan los `grace_period_days` del plan (v3.2) → `access.status` pasa a `paused_unpaid`.
3. El alumno sigue viendo su perfil y sus notas, pero no el contenido, y ve un aviso para regularizar.
4. Al registrarse el pago → vuelve a `active` **sin perder el progreso ni los desbloqueos** (`content_unlocks`).
5. El reloj de `learning_enrollments`: la política por defecto es que **sigue corriendo** durante la pausa. Es un setting editable por organización (`pause_clock_on_unpaid`), porque es una decisión comercial.

Todo esto se dispara desde el Event Bus interno (v3.2, 14.5): el gateway avisa, se traduce a `PaymentApproved` o `PaymentFailed`, y el módulo de accesos reacciona.

---

9. Dominio nuevo: Gateways, reparto y pagos a autores

9.1 Cuentas de cobro

`gateway_accounts`: cuentas del gateway por organización (la de LA TROUPE hoy; las de otras academias mañana).

Campo	Tipo	Notas
id	UUID	
organization_id	UUID	**Cada academia cobra en su propia cuenta**
gateway	enum	`mercadopago`, `stripe`
external_account_id	string	
credentials_ref	string	Referencia a secretos cifrados; **nunca** las claves en la tabla
status	enum	`active`, `disconnected`
supported_currencies	string[]	

9.2 Cuentas de cobro de los autores

`payout_accounts`: la cuenta donde el profesor recibe su parte.

Campo	Tipo	Notas
id	UUID	
user_id	UUID	El profesor
organization_id	UUID	
method	enum	`mercadopago_connect`, `stripe_connect`, `bank_transfer`

Campo	Tipo	Notas
external_account_id	string	
status	enum	`pending`, `connected`, `disconnected`
connected_at	timestamp	

El profesor conecta su cuenta con un flujo de autorización (OAuth) desde su panel. La academia nunca ve sus credenciales.

9.3 Acuerdos de reparto

`revenue_share_agreements`: qué cobra cada autor, por curso o en general, con vigencia en el tiempo.

Campo	Tipo	Notas
id	UUID	
organization_id	UUID	
beneficiary_user_id	UUID	El profesor
scope	enum	`all_courses` o `course` (con `course_id`)
model	enum	Ver tabla abajo
platform_fee_pct	decimal	Comisión de la academia
fixed_amount	decimal	Para `fixed_fee` y `hybrid`
payout_mode	enum	`automatic_split` o `manual_payout`
valid_from / valid_to	date	Vigencia. Un nuevo acuerdo no cambia el pasado
status	enum	`draft`, `active`, `ended`

model	Significa
`percentage`	El profesor cobra un porcentaje de cada venta; la academia se queda con la comisión
`fixed_per_sale`	Monto fijo por cada alumno que entra
`fixed_fee`	La academia cobra todo y paga una suma pactada (sin relación con las ventas)
`hybrid`	Parte fija más porcentaje

9.4 Los dos caminos de pago al profesor

Modo	Cómo funciona	Para quién
`automatic_split`	Al cobrar al alumno, el gateway divide el pago: la parte del profesor va directo a su `payout_account` y la comisión queda en la cuenta de la academia	Profesores invitados que cobran por porcentaje
`manual_payout`	Todo entra a la cuenta de la academia. El sistema calcula lo que se debe y la academia paga (usa `teacher_compensation_rules` y `teacher_payouts` de la v3.2)	Profesores con monto fijo o relación laboral

Los dos modos **pueden convivir** en la misma academia, un profesor por acuerdo. Qué modo y qué porcentaje tiene cada profesor se define y se cambia desde el panel de administración.

> **Para verificar en la fase técnica.** La división automática depende de las capacidades de cada gateway (MercadoPago marketplace con comisión, Stripe Connect). Hay que confirmar disponibilidad por país, requisitos de verificación de identidad del profesor, cómo se manejan reembolsos y contracargos sobre un pago ya dividido, y quién responde por ellos. El modelo de datos no depende de esa respuesta, pero el flujo sí.

9.5 Registro del reparto

`payment_splits`: una fila por beneficiario por pago.

Campo	Tipo	Notas
id	UUID	
payment_id	UUID	
beneficiary_type	enum	`author`, `platform`, `organization`
beneficiary_user_id	UUID	
agreement_id	UUID	Qué acuerdo se aplicó
gross_amount	decimal	
fee_amount	decimal	Comisión descontada
net_amount	decimal	
currency	string	
status	enum	`pending`, `transferred`, `held`, `refunded`, `failed`
external_transfer_id	string	Referencia del gateway
transferred_at	timestamp	

Con esto, "cuánto vendió este profesor este mes y cuánto le corresponde" es una consulta, no un cálculo reconstruido. En un reembolso se agregan filas negativas; las originales no se tocan (principio 2.14).

9.6 Dos capas de comisión

Capa	Quién cobra	A quién	Tabla
Comisión de la academia al profesor invitado	LA TROUPE (o cada academia)	Profesores	`revenue_share_agreements`
Licencia de la plataforma a las academias (futuro)	El equipo de desarrollo	Organizaciones	`platform_subscriptions` (v3.2)

La segunda capa no se construye en el MVP. LA TROUPE no se cobra a sí misma.

9.7 Varias monedas y países

- `prices.currency` y `payments.currency` son obligatorios en todo pago; no hay moneda "por defecto" escondida en el código.
- El gateway se elige por país del comprador (v3.2, 14.5): MercadoPago para Latinoamérica, Stripe para España y resto del mundo.
- `tax_amount` y `tax_inclusive` se reservan en precios y pagos. Las reglas de IVA y fiscalidad de venta digital a la UE las define un contador **antes de lanzar en España**; no se programan hasta entonces.

---

10. Dominio nuevo: Licencias para academias (contenido B2B)

Una academia compra contenido para usarlo en sus clases presenciales: planificaciones, coreografías, material para el profesor.

`licenses`

Campo	Tipo	Notas
id	UUID	

Campo	Tipo	Notas
product_id	UUID	
licensee_organization_id	UUID	La academia compradora
seats	integer	Cuántas personas pueden usarlo (null = ilimitado)
valid_from / valid_until	date	
terms	text	Alcance de uso
status	enum	`active`, `expired`, `revoked`
access_id	UUID	El `access` que genera, con `grantee_type = organization`

Los contenidos para academias son cursos con `audience = ['academy_b2b']` y bloques de tipo `download` (planificaciones, guías en PDF). Los profesores de la academia compradora ingresan con su cuenta y ven el material a través del acceso de su organización.

Esto se diseña ahora y se construye después. Lo único que no se puede dejar para más tarde es que `access` admita una organización como titular (sección 8.3).

---

11. Roadmap rehecho

El orden se basa en lo que necesita el negocio primero: **vender cursos on demand con perfil de alumno, panel de gestión y control de avance**. Las clases presenciales y en vivo pasan a una fase posterior.

Fase 0 — Cimientos (antes de la primera pantalla)

- Repositorio, entornos, migraciones, CI.
- Proyecto en Supabase con Row-Level Security activo desde el inicio.
- Esquema de identidad, roles y permisos (sección 3), con los roles de fábrica.
- Pruebas automáticas de aislamiento entre organizaciones y de permisos.
- **Reservar** las columnas que no se pueden agregar después sin dolor (autor, estado de revisión, titular polimórfico de `access`, modalidad de cobro del precio, `pacing_mode`).

Fase 1A — Producto mínimo vendible

- Registro e inicio de sesión (email y Google); perfil de alumno; "Mis cursos".
- Catálogo público con tráiler y clip al pasar el mouse.
- Carreras, cursos, lecciones y bloques (video, texto, diapositivas, descargas).
- Panel de administración: cursos, carreras, lecciones, alumnos, accesos, pagos.
- Cobro con MercadoPago: pago único, cuotas con tarjeta y suscripción recurrente.
- Acceso según compra y estado del pago.
- Liberación de contenido: libro abierto y curricular **por tiempo**.
- Reproductor con velocidad, capítulos y modo espejo.
- Captura de datos desde el día uno (progreso, visualizaciones, eventos).
- Reporte básico de ingresos y alumnos.

Fase 1B — Evaluación y egreso

- Evaluaciones automáticas y mixtas; calificación manual por el docente.
- Liberación por examen y combinada (tiempo y examen).
- Reglas de egreso, aprobación manual y certificados con verificación pública.

Fase 2 — Profesores y reparto

- Portal del profesor; flujo de revisión y aprobación; solicitudes de cambio.
- Cuentas de cobro de los profesores y reparto automático.
- Pago manual para profesores con monto fijo.
- Reportes por profesor.
- Plan de pagos propio (N cobros con acceso condicionado).

Fase 3 — Mercado internacional

- Stripe, multimoneda y enrutamiento de pagos por país.
- Reglas fiscales según el contador.

Fase 4 — Licencias para academias

- Productos tipo licencia, contenido B2B, acceso por organización.

Fase 5 — Retención, analítica y app móvil

- "Continuá donde quedaste", tableros del owner, reportes de comportamiento.
- App móvil, WhatsApp, Loop A-B.

Fase 6 — Clases en vivo y presenciales (de la v3.2, Fase 3)

- `class_sessions`, inscripciones a clases, asistencia, lista de espera, Zoom.

Fase 7 — SaaS multi-organización

- Panel de plataforma, alta de academias, selector de organización, marca propia, dominio propio, licencias de la plataforma.

> Los plazos en semanas de la v3.2 estaban pensados para dos desarrolladores. Para este proyecto no se estiman hasta definir el ritmo de trabajo real; la secuencia de fases sí es firme.

---

12. Qué se reserva ahora y qué se crea después

La regla (1.2): lo caro de cambiar se reserva; lo independiente se crea cuando toca.

Qué	Cuándo
Columnas en `courses`: `author_user_id`, `owner_type`, `audience`, `review_status`, `pacing_mode`, tráiler y clip	**Fase 0/1A**
`access` con titular polimórfico, `product_id`, `status`	**Fase 0/1A**
`products`, `product_items`, `prices` con `billing_model` y `access_policy`	**Fase 1A**
`roles`, `permissions`, `role_permissions`, `membership_roles`	**Fase 0**
`programs`, `program_courses`, `lesson_blocks`, `learning_enrollments`, `release_rules`, `content_unlocks`	**Fase 1A**
`assessments` y tablas asociadas, `completion_rules`, `completions`, `certificate_templates`, `certificates`	**Fase 1B**
`gateway_accounts`	**Fase 1A** (la de LA TROUPE)

Qué	Cuándo
`payout_accounts`, `revenue_share_agreements`, `payment_splits`, `course_change_requests`	**Fase 2** (las columnas de apoyo en `courses` y `payments` ya están)
`licenses`	**Fase 4**
Tablas de clases en vivo	**Fase 6**
`platform_subscriptions`	**Fase 7**

Conteo: la v3.2 tenía 27 tablas; la v3.3 suma unas 27 nuevas y reemplaza o modifica 6. Son muchas, pero se crean por fase y ninguna de las posteriores obliga a migrar datos de las anteriores.

---

13. Casos de uso nuevos

13.1 Carrera de un año con liberación mensual por tiempo y examen

1. Julia se inscribe en junio a 'Formación Urbano — Año 1' (`curricular`). Su `learning_enrollments.started_at` es junio.
2. El módulo 1 está abierto. Los módulos 2 a 12 tienen una `release_rule`: `all` de *un mes desde la inscripción* y *aprobar el examen del módulo anterior*.
3. A los 30 días de su inscripción se abre la ventana mensual del examen del módulo 1.
4. Julia lo rinde. Es `mixed`: una parte se corrige sola y un video de práctica lo califica la docente.
5. Al quedar `graded` y `passed`, se evalúa la regla del módulo 2: ya pasó el mes y aprobó → se escribe un registro en `content_unlocks` y el módulo 2 aparece.
6. Pedro, que se inscribió en enero, está en el módulo 6. Mismo contenido, otro ritmo.

13.2 Profesor invitado que vende su curso

1. La academia invita a Marco y le asigna el rol *Profesor invitado*. Marco conecta su cuenta de MercadoPago (`payout_accounts`).
2. Se firma un acuerdo (`revenue_share_agreements`): 70 % para Marco, 30 % para la academia, `automatic_split`.
3. Marco crea su curso en `draft`, sube las lecciones y lo envía a revisión.
4. Un revisor lo aprueba. Desde ese momento el contenido queda congelado.
5. Una alumna compra por 100 USD: el gateway reparte, el sistema crea dos filas en `payment_splits` (70 y 30).
6. Marco quiere reemplazar una lección: crea una `course_change_request`. Un revisor la aprueba y recién ahí se aplica.

13.3 Alumno con plan de pagos que deja de pagar

1. Luis eligió un plan de 12 cobros mensuales con acceso `while_paid`.
2. En el cobro 5 falla la tarjeta. Pasan los días de gracia.
3. Su `access` pasa a `paused_unpaid`. Mantiene su perfil, su historial y sus notas, pero no ve el contenido.
4. Paga la cuota atrasada: el acceso vuelve a `active`. Sus desbloqueos y su progreso siguen intactos.

13.4 Academia externa compra material para sus clases presenciales

1. Academia Delta compra una licencia de 'Planificación anual — Niveles iniciales' (producto `license`, 3 usuarios).
2. Se crea una `license` y un `access` cuyo titular es la organización Delta.
3. Las tres profesoras de Delta ingresan con sus cuentas y acceden a los PDF y videos del curso.

---

14. Riesgos y puntos a validar

Tema	Riesgo	Qué hacer
Reparto automático	Depende de capacidades y reglas de cada gateway; los reembolsos sobre pagos ya divididos son complejos	Prueba de concepto en sandbox antes de la Fase 2
Plan de pagos propio	Cobros programados, reintentos y conciliación son lo más delicado del sistema	Construirlo después de que suscripción y pago único estén estables
IVA y fiscalidad en España	Reglas propias para servicios digitales	Contador antes de lanzar
Datos de menores	Hay alumnos menores de edad; notas, salud y fotos son datos sensibles	Revisar con asesoría legal los requisitos de consentimiento y almacenamiento
Clip al pasar el mouse	La capacidad de Bunny.net para esto no está confirmada	Verificar en la Fase 0
Protección de video	HLS cifrado frena la copia casual, no la determinada	Aceptado como decisión (v3.2, 14.4)
Complejidad	El volumen de tablas es alto	Crear por fase; pruebas automáticas de permisos y de reglas de liberación desde el inicio
Reloj durante la pausa por impago	Decisión comercial sin definir	Setting `pause_clock_on_unpaid`, a decidir por el owner

---

15. Decisiones tomadas en esta versión

Tema	Decisión
Modalidades de avance	Libro abierto y curricular; el curricular admite tiempo, examen, o ambos (`all` / `any`), con reloj individual por alumno
Calificación	Automática, manual o mixta, configurable por evaluación
Egreso	Por criterios automáticos, con aprobación manual opcional, o solo manual; certificado verificable
Modalidades de cobro	Pago único, cuotas con tarjeta, suscripción y plan de pagos propio: las cuatro modeladas desde el inicio
Gateways	MercadoPago primero, Stripe en Fase 3; cada academia cobra en su propia cuenta
Pago a profesores	Automático por división de pago, o manual; ambos conviven y se definen por profesor
Contenido para academias	Se modela como licencia con titular organización; se construye en Fase 4
Público del contenido	Etiqueta `audience` en cursos y productos
Permisos	Roles y permisos editables; reemplazan los tres enums
Clases en vivo y presenciales	Pasan a la Fase 6

15.1 Pendientes de definir (no bloquean la Fase 0)

- Tipos concretos de suscripciones y precios.
- Si el reloj de un alumno sigue corriendo cuando su acceso está pausado por impago.
- Porcentajes de comisión por tipo de profesor.
- Estructura de certificados (diseño, firmantes).
- Política de reembolsos para cursos on demand.
- Edad mínima para cuenta propia y consentimiento de padres.

---

16. Próximos pasos

1. Revisar este documento y marcar lo que no coincida con la idea.
2. Resolver el prototipo de Bunny.net (clip al pasar el mouse, protección y reproducción).
3. Definir el esquema SQL de la Fase 0 y las políticas de Row-Level Security.
4. Diseñar las pantallas críticas de la Fase 1A: catálogo, ficha del curso, checkout, "Mis cursos", reproductor, panel de cursos y de alumnos.
5. Abrir las cuentas en modo de prueba: Supabase, MercadoPago, Bunny.net, Vercel, Resend.